



EGE UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

569406 PROGRAMMING LANGUAGES

2024–2025 SPRING SEMESTER

PROJECT-2 REPORT

(Interpreter for the Plus++ Language)

DELIVERY DATE

15/06/2025

PREPARED BY

05230000350, Ceylin TOR

05220001067, Aykut HACIOĞLU

05220000366 , Lidya DELİAĞA

05230000983 , Ali Batıkan ÖCAL

Contents

1) PROJECT DEFINITION AND EXPLANATION	2
2) CODE DESCRIPTION	4
2.a) Libraries & Constants	4
2.b) Program Storage in Memory	4
2.c) Variable Management	5
2.d) Utility Functions	6
2.e) Parsing Strategy	7
2.f) Single-Line Interpreter	8
2.g) Repeat Block Execution	10
2.h) Main Function	13
3) ERROR HANDLING.....	14
3.a) Undeclared Variable.....	14
3.b) Invalid Variable Name	15
3.c) Missing Semicolon	15
3.d) Invalid Assignment Format.....	16
3.e) Missing { in Repeat Block.....	16
3.f) Missing } in Repeat Block.....	17
3.g) Invalid Number Format	17
4) NO ERROR EXAMPLES	18
4.a) myscript.ppp.....	18
4.b) test_assign.ppp.....	19
4.c) test_math.ppp	19
4.d) test_nested.ppp.....	20
4.e) test_repeat.ppp	20
4.f) test_write.ppp	21

1) PROJECT DEFINITION AND EXPLANATION

This project implements an **interpreter for the Plus++ programming language** using the C programming language. It reads .ppp source files and executes the instructions they contain. The interpreter supports variables, arithmetic operations, nested repeat blocks, and structured error reporting.

Purpose

The purpose of this project is to:

- Parse and execute Plus++ programs with basic control flow.
- Track declared variables and their values during execution.
- Handle syntax errors and runtime issues such as undeclared or uninitialized variables.

Functionality

The analyzer performs the following key tasks:

- **File Reading:** Loads all lines of the .ppp file into memory.
- **Variable Management:** Handles variable declarations using number. Keeps track of their values, declaration, and initialization status.
- **Assignments & Arithmetic:** Supports :=, +=, and -= operators. Right-hand values can be constants or other variables.
- **Write Statements:** Prints strings, variable values, and line breaks using write ... and ... and newline;.
- **Repeat Loops:** Executes code blocks multiple times using repeat N times { ... }. Nested loops are supported with proper brace matching.
- **Error Handling:** Detects undeclared or uninitialized variables, invalid syntax, missing semicolons, and unmatched {} blocks, with line-specific error messages.

2) CODE DESCRIPTION

2.a) Libraries & Constants

```
#include <stdio.h> //printf, fgets işlemleri
#include <stdlib.h> //exit, atoll
#include <string.h> //strcpy, strcat gibi string işlemleri
#include <ctype.h> //isspace, isdigit, isalpha gibi karakter kontrolleri

#define MAX_VARS 256
#define MAX_LINE 1024 //satir max ch
#define MAX_NAME 21 //20 karakter + null
#define MAX_PROGRAM_LINES 1000
```

- **Included Libraries:**

- <stdio.h> — Input/output functions like printf, fgets.
- <stdlib.h> — Utilities like exit, atoll.
- <string.h> — String manipulation (e.g., strcpy, strcat).
- <ctype.h> — Character checks (e.g., isspace, isdigit).

- **Constants Defined:**

- MAX_VARS, MAX_LINE, MAX_NAME, MAX_PROGRAM_LINES
These define limits for the number of variables, line size, and program length.

2.b) Program Storage in Memory

```
// .ppp dosyasını bellekte tutan matris
char all_program_lines[MAX_PROGRAM_LINES][MAX_LINE];
int total_program_lines = 0;
```

The interpreter stores each line of the .ppp file in a 2D array. This lets the interpreter access and process each line individually – useful for handling loops and nested blocks. total_program_lines keeps track of how many lines were loaded.

2.c) Variable Management

```
// değişkenin bilgilerini tutan yapı
typedef struct {
    char name[MAX_NAME];
    char declared;
    char initialized;
    long long value;
} Variable;

// değişken listesi
Variable vars[MAX_VARS];
int var_count = 0;
```

```
//Değişken bulma
Variable* find_var(const char *name) {
    for (int i = 0; i < var_count; i++) {
        if (strcmp(vars[i].name, name) == 0)
            return &vars[i];
    }
    return NULL;
}

// degisken tanımlama
void declare_var(const char *name, int line_num) {
    // degisken var mi yok mu kontrol
    if (find_var(name)) {
        return;
    }
    // degisken limit kontrolu
    if (var_count >= MAX_VARS) {
        printf("Error on line %d: Too many variables (max %d).\n", line_num, MAX_VARS);
        exit(1);
    }
    //yeni degiskene baslangis degeri verme
    strncpy(vars[var_count].name, name, MAX_NAME - 1);
    vars[var_count].name[MAX_NAME - 1] = '\0';
    vars[var_count].value = 0; // deger 0
    vars[var_count].declared = 1;
    vars[var_count].initialized = 1; // başlangıç değeri verildi 1 oldu
    var_count++;
}

//değer atama
void set_var(const char *name, long long value, int line_num) {
    Variable *var = find_var(name);
    if (!var) {
        printf("Error on line %d: Undeclared variable '%s'.\n", line_num, name);
        exit(1);
    }
    var->value = value;
    var->initialized = 1;
}
```

```

// değer okuma
long long get_var(const char *name, int line_num) {
    Variable *var = find_var(name);
    if (!var) {
        printf("Error on line %d: Undeclared variable '%s'.\n", line_num, name);
        exit(1);
    }
    if (!var->initialized) {
        printf("Error on line %d: Uninitialized variable '%s'.\n", line_num, name);
        exit(1);
    }
    return var->value;
}

```

Variable: A struct that holds the name, declaration status, initialization status, and current value of a variable.

vars[]: Stores all declared variables.

var_count: Tracks how many variables have been declared.

find_var: Searches the vars[] array for a variable by name. Returns a pointer to the variable if found; otherwise, returns NULL.

declare_var: Declares a new variable if it hasn't already been declared. Checks the maximum variable limit (MAX_VARS). Initializes the variable with default values: values = 0, declared = 1, initialized = 1.

set_var: Assigns a new value to an existing variable. If the variable is undeclared, an error message is printed and the program exits. Marks the variable as initialized.

get_var: Returns the value of a declared and initialized variable. If the variable is undeclared or uninitialized, it reports an error.

2.d) Utility Functions

```

// bosluklari siler
void trim(char *str) {
    int i = 0, j = 0;
    while (isspace((unsigned char)str[i])) i++;
    while (str[i]) str[j++] = str[i++];
    str[j--] = '\0';
    while (j >= 0 && isspace((unsigned char)str[j])) str[j--] = '\0';
}

// Satır boş mu veya yorum mu
int is_comment_or_empty(const char *line) {
    return (line[0] == '\0' || line[0] == '*');
}

```

```

// degisken adinin gecerliligi kontrol ediliyor
int is_valid_name(const char *name) {
    if (!name || *name == '\0') return 0; // ad bos olamaz
    if (!isalpha((unsigned char)name[0])) return 0; // harf ile baslamali
    for (int i = 1; name[i]; i++) {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_') return 0; // harf sayi altcizgi olabilir
    }
    return 1;
}

// gecerli sayi olup olmadigi kontrol edilir
int is_valid_number(const char *val) {
    if (!val || *val == '\0') return 0; // bos olamaz
    int i = 0;
    if (val[0] == '-') { // onde gelen - izin verilir
        i++;
        if (val[1] == '\0') return 0; // "-" tek basina gecerli sayi degil
    }
    // sonraki karakter sayi olmalı
    for (; val[i]; i++) {
        if (!isdigit((unsigned char)val[i])) return 0;
    }
    return 1;
}

```

- **trim(char *str):** Removes leading and trailing whitespace from a string. This ensures that input lines and tokens are clean before processing, preventing errors caused by unnecessary spaces.
- **is_comment_or_empty(const char *line):** Checks whether a line is empty or a comment. If the line is either blank or starts with *, it is skipped during interpretation.
- **is_valid_name(const char *name):** Validates variable names. A valid name must start with a letter and can contain letters, digits, or underscores. It ensures identifiers follow Plus++ naming rules.
- **is_valid_number(const char *val):** Checks if a string represents a valid integer. It allows optional leading minus signs and ensures all characters are digits, rejecting invalid number formats.

2.e) Parsing Strategy

The interpreter parses each line manually using basic string functions. It detects keywords (number, write, repeat) and operators (:=, +=, -=) with strncmp and strstr. Assignments are parsed using sscanf to extract variable names and values. Instead of splitting the line into formal tokens, the interpreter directly checks patterns and processes them.

For example, in write statements, it scans character by character to handle strings, variables, and, and newline. Loop blocks use brace counting to track nested structures.

2.f) Single-Line Interpreter

```
// tek satirlik kodlari yorumlar repeat haric
void interpret_single_line(const char *line, int line_num) {
    // noktali virgul kontrolu
    if ((strcmp(line, "number", 6) == 0 ||
        strstr(line, ":=") != NULL ||
        strstr(line, "+=") != NULL ||
        strstr(line, "-=") != NULL ||
        strcmp(line, "write", 5) == 0) && strchr(line, ';') == NULL) {
        printf("Syntax error on line %d: Missing ';' at end of statement.\n", line_num);
        exit(1);
    }

    // degisken tanimlama
    if (strcmp(line, "number", 6) == 0) {
        char name[MAX_NAME];
        if (sscanf(line + 6, "%20[^\n];", name) == 1 && is_valid_name(name)) {
            trim(name);
            declare_var(name, line_num); //tanimlama
            // sscanf başarısızsa ya da isim geçerli değilse
        } else {
            printf("Syntax error on line %d: Invalid 'number' declaration.\n", line_num);
            exit(1);
        }
    }

    // atama islemi
    } else if (strstr(line, ":=") != NULL) {
        char var[MAX_NAME], val_str[MAX_LINE]; //solda değişkenin ismi sagda atanan deger
        if (sscanf(line, "%20[^:]:= %255[^\n];", var, val_str) == 2) { // satırı parçalar
            // := operatöründen önceki kısmı var değişkenine atar
            // ; işaretine kadar olan kısmı val_str'a atar
            trim(var); trim(val_str); //bosluklari temizler
            if (!is_valid_name(var)) {
                printf("Error on line %d: Invalid variable name '%s' in assignment.\n", line_num, var);
                exit(1);
            }
            if (is_valid_number(val_str)) { //atanacak deger sayi ise
                set_var(var, atoll(val_str), line_num); // atoll ile long long değere çevrilir set_var ile atanır
            } else if (is_valid_name(val_str)) {
                set_var(var, get_var(val_str, line_num), line_num); //değişkenin get var ile değeri alınır ve vine set var ile atanır
            }
        }
    }
```

```
    } else {
        printf("Error on line %d: Invalid value '%s'. Must be a number or declared variable.\n", line_num, val_str);
        exit(1);
    }

    } else {
        printf("Syntax error on line %d: Invalid assignment statement.\n", line_num);
        exit(1);
    }

    //arttırma islemi
    } else if (strstr(line, "+=") != NULL) {
        char var[MAX_NAME], val_str[MAX_LINE];
        if (sscanf(line, "%20[^+=] += %255[^\n];", var, val_str) == 2) { // satırı parçalar

            trim(var); trim(val_str); //bosluklari temizler
            long long current_val = get_var(var, line_num); //var değişkeninin mevcut değeri alınır.
            if (is_valid_number(val_str)) { //atanacak deger sayiysa
                set_var(var, current_val + atoll(val_str), line_num);
            } else { //Eğer sağ taraf sayı değilse degiskenin get_var ile degeri alınır
                set_var(var, current_val + get_var(val_str, line_num), line_num);
            }
            // sscanf(...) başarısız olursa
        } else {
            printf("Syntax error on line %d: Invalid '+' assignment.\n", line_num);
            exit(1);
        }
    }

    // azaltma islemi
    } else if (strstr(line, "-=") != NULL) {
        char var[MAX_NAME], val_str[MAX_LINE];

        if (sscanf(line, "%20[^-=] -= %255[^\n];", var, val_str) == 2) {
            trim(var); trim(val_str);
```



```

        long long current_val = get_var(var, line_num);
        if (is_valid_number(val_str)) {
            set_var(var, current_val - atoll(val_str), line_num);
        } else {
            set_var(var, current_val - get_var(val_str, line_num), line_num);
        }
    } else {
        printf("Syntax error on line %d: Invalid '-' assignment.\n", line_num);
        exit(1);
    }
}

// satir write ile baslayip baslamadiginin kontrolu
} else if (strncmp(line, "write ", 6) == 0) {
    char *p = (char *)line + 6; // geri kalan içeriği işlemek için p pointeri olusturduk
    char token_buf[MAX_LINE]; // gecici buffer degisken ismi burda tutulur
    int in_string = 0; // string kontrolu icin flag

    while (*p) { // ch ch okunur
        while (isspace((unsigned char)*p)) p++; // bosluklari atlamak icin

        if (*p == '"') { // string baslangici
            p++;
            while (*p && *p != '"') {
                putchar(*p); // string icindekileri ekrana yazdirir
                p++;
            }
            if (*p == '"') p++; // " bu varsa bunu gec
        } else if (strncmp(p, "newline", 7) == 0 && (isspace((unsigned char)p[7]) || p[7] == ';' || p[7] == '\0')) {
            //newline komutuysa ekrana yeni satir yazdirilir p ileri alinir >> 7
            printf("\n");
            p += 7;
        } else if (strncmp(p, "and", 3) == 0 && isspace((unsigned char)p[3])) {
            //andden sonra bosluk varsa baglactir bisi yapilmaz and gecilir
            p += 3;
        } else if (isalpha((unsigned char)*p)) { // harfle baslamasi gerekiyor
            int i = 0;

```

```

            while (isalnum((unsigned char)*p) || *p == '_' ) { // harf sayi _ ile devam edebilir
                token_buf[i++] = *p;
                p++;
            }
            token_buf[i] = '\0';
            // variable gecersizse error yazdirilir
            if (!is_valid_name(token_buf)) {
                printf("Error on line %d: Invalid variable name '%s' in write statement.\n", line_num, token_buf);
                exit(1);
            }

            printf("%lld", get_var(token_buf, line_num)); // Write variable value
        } else {
            // beklenmeyen karakter ise atlanir
            p++;
        }
    }
    printf("\n"); // her write komutu sonrasi yeni satir eklenir
    // operator string write kontrolu vs degilse error
} else {
    printf("Syntax error on line %d: Unrecognized statement '%s'.\n", line_num, line);
    exit(1);
}
}

```

interpret_single_line(...) handles all non-repeat statements. Checks for syntax correctness, valid identifiers and values, missing semicolons. Invalid statements cause immediate error messages with line numbers.

- **Semicolon Check:** The function first checks if the line is a number, assignment, or write statement, and ensures it ends with a semicolon. If not, it raises a syntax error.

- **Variable Declaration (number):** If the line starts with number, it extracts the variable name, trims it, checks if it's valid, and then declares it using `declare_var`. If invalid, it throws a syntax error.
- **Assignment (:=):** Lines with `:=` are treated as assignment statements. It separates the variable and value, trims both, validates them, and assigns the value — either a number or another variable — using `set_var`.
- **Increment (+=):** For `+=`, it extracts the variable and value, fetches the current variable's value, adds the right-hand side (number or variable), and updates the result with `set_var`.
- **Decrement (-=):** The `-=` case works like `+=`, but performs subtraction instead. It ensures valid input and updates the variable accordingly.
- **Write Statement:** If the line starts with `write`, it processes and prints strings, variables, or newline, handling them with `and` as a connector. Variable names are validated before printing their values.
- **Unrecognized Statement:** If the line does not match any known pattern, the function reports a syntax error with the full line content and exits the program.

2.g) Repeat Block Execution

```
// all_program_lines dizisindeki start_idx ile end_idx aralığındaki satırları işler
// base_line_num gerçek satır numarası (hata mesajları doğru gözüksün diye)

void execute_program_block(int start_idx, int end_idx, int base_line_num) {
    int current_relative_line = 0; //bloğun içinde kaç satır ilerlediğimizi sayar counter ve hata mesajlarında satır sayısı

    for (int i = start_idx; i < end_idx; i++) {
        char line_copy[MAX_LINE];
        strcpy(line_copy, all_program_lines[i]);
        trim(line_copy);

        int current_actual_line_num = base_line_num + current_relative_line; // bu satırın dosya üzerindeki sıra numarasını hesaplar

        if (is_comment_or_empty(line_copy)) { //satır bos veya yorumsa atla
            current_relative_line++;
            continue;
        }

        //repeat varsa
        if (strncmp(line_copy, "repeat ", 7) == 0) {
            char counter_str[MAX_NAME]; //tekrar sayısını geçici olarak saklamak için
            long long repeat_value;
            char *ptr = line_copy + 7; //repeati atlat
            while (*ptr == ' ') ptr++; //bosluklari atla
            sscanf(ptr, "%20[^ ]", counter_str); // repeatten sonra geleni counter_str icine alir
            trim(counter_str);

            if (is_valid_number(counter_str)) {
                repeat_value = atoll(counter_str); //counter_str deki degeri sayiya çevirerek bu degişkene atariz
            } else {
                repeat_value = get_var(counter_str, current_actual_line_num); //degişken adıyla o degişkenin degerini alir
            }

            //{} aralığını tanımlamak
            int block_start_idx = i + 1;
            int block_end_idx = -1;
            int brace_level = 0;

            // { aynı satırdaysa
```

```

if (strchr(line_copy, '{') != NULL) {
    brace_level = 1; // { gordugu için 1 oldu ilerde {} esitligi için kullanılacak
    block_start_idx = i + 1; // block icerigi sonraki satirda baslatiliyor
} else {
    // sonraki satirlarda { araniyor
    int j = i + 1;
    while (j < end_idx && j < total_program_lines) { // kodun sınırlarını aşmamak için
        char temp_line[MAX_LINE];
        strcpy(temp_line, all_program_lines[j]);
        trim(temp_line);
        current_relative_line++; //bu satir islendi bir artti
        if (strcmp(temp_line, "{") == 0) { //trimlenen satirda { kaldiyse blok baslar
            brace_level++;
            block_start_idx = j + 1; // block icerigi sonraki satirda baslatiliyor
            break;
        }
        j++;
    }
    // { hiç bulunamazsa hata verilir.
    if (brace_level == 0) {
        printf("Syntax error on line %d: Missing opening '{' for repeat block.\n", current_actual_line_num);
        exit(1);
    }
}

// {} dengesi
int j = block_start_idx;
while (j < end_idx && j < total_program_lines) { // kodun sınırlarını aşmamak için
    char temp_line[MAX_LINE];
    strcpy(temp_line, all_program_lines[j]);
    trim(temp_line);
    current_relative_line++; //bu satir islendi bir artti

    if (strcmp(temp_line, "{") == 0) { //trimlenen satirda { kaldiyse blok baslar
        brace_level++; // Nested {
    } else if (strcmp(temp_line, "}") == 0) {
        brace_level--; // esitlik için azaltilir
        if (brace_level == 0) {

```

```

            block_end_idx = j; //bitis blogu } indexi
            break;
        }
        j++;
    }
    //parantez dengesi bozulmuşsa veya hic } bulunmamissa error
    if (brace_level != 0 || block_end_idx == -1) {
        printf("Syntax error on line %d: Missing closing '}' for repeat block.\n", current_actual_line_num);
        exit(1);
    }

    // döngünün kaç defa tekrar edeceği
    for (long long k = repeat_value; k > 0; k--) {
        // Sayaç bir değişken adıysa recursive için değeri güncellenir
        if (!is_valid_number(counter_str)) {
            set_var(counter_str, k, current_actual_line_num);
        }
        execute_program_block(block_start_idx, block_end_idx, current_actual_line_num + 1); //belirlenen blok içerisindeki satırlar arasına
    }
    // degiskense döngü bittikten sonra değeri sıfırlanır
    if (!is_valid_number(counter_str)) {
        set_var(counter_str, 0, current_actual_line_num);
    }

    // i '}' satırına kadar atlatılır
    i = block_end_idx;
} else {
    // satir repeat bloku degilse interpret_single_line(...) çağrılır
    interpret_single_line(line_copy, current_actual_line_num);
}
current_relative_line++; // satir sayısını izlemek için sayac artar
}
}

```

execute_program_block is responsible for executing a block of Plus++ code line by line, including handling nested repeat loops. It processes lines between the given start and end indices (start_idx to end_idx) and tracks the original line number for accurate error reporting.

- **Line Processing:** Each line is copied and trimmed.
The `current_relative_line` counter tracks the interpreter's progress within the block and adjusts line numbers for accurate error reporting.
- **Comment and Empty Line Skipping:** Lines that are empty or start with `*` are treated as comments and skipped during execution.
- **Detecting repeat Statements:** If a line starts with `repeat`, the loop counter is extracted. It can be a literal number or a variable. The interpreter then searches for the opening brace `{`, which can appear on the same line or in subsequent lines.
- **Brace Matching and Nesting:** The function uses a `brace_level` counter to track `{` and `}` pairs. This allows proper handling of nested repeat blocks. If matching braces are not found, a syntax error is raised.
- **Loop Execution:** After identifying the block, the interpreter runs it as many times as the loop counter specifies. The inner block is executed recursively using `execute_program_block`.
- **Line Number Accuracy in Loops:** The recursive call passes `current_actual_line_num + 1` to preserve accurate line numbers within the loop body when reporting errors.
- **Resetting the Loop Counter:** If the loop counter is a variable, its value is updated during each iteration and reset to 0 after the loop ends to avoid affecting later code.
- **Regular Line Execution:** Lines that are not part of a repeat are passed to `interpret_single_line` for processing (e.g., declarations, assignments, write statements).
- **Safety Checks:** To avoid reading beyond the file, the function checks that line indices do not exceed `total_program_lines`.
- **Brace Flexibility:** The interpreter allows the opening `{` of a repeat block to appear on the same line as the repeat statement, such as `repeat 3 times {`.

2.h) Main Function

```
//program çalışmaya burada baslar
int main(int argc, char *argv[]) { //komut satırından girilen argüman bilgileri
    char filename[256];
    if (argc == 2) { // örneğin ./ppp myscript gibi çağırınca .ppp yi kendi ekler
        snprintf(filename, sizeof(filename), "%s.ppp", argv[1]);
    } else { // komut satırına dosya ismi girilmezse
        printf("Enter filename (without .ppp): ");
        if (scanf("%255s", filename) != 1) { // girdi alınır
            fprintf(stderr, "Error reading filename.\n");
            return 1;
        }
        strcat(filename, ".ppp"); // .ppp uzantısı eklenir
    }

    FILE *file = fopen(filename, "r");
    if (!file) { //eğer dosya açılamazsa error verir ve çıkar
        perror("Error opening file");
        return 1;
    }

    // Read the entire file into memory
    int current_line_in_file = 0;
    //her satır diziye aktarılır ve her bir satır bittikten sonra sonuna \0 ekler
    while (fgets(all_program_lines[current_line_in_file], sizeof(all_program_lines[0]), file)) {
        trim(all_program_lines[current_line_in_file]); //her satır okunduğu gibi boşluklar temizlenir
        current_line_in_file++; //satır sayacı
        if (current_line_in_file >= MAX_PROGRAM_LINES) { //satır sayısı sınırı(1000) geçerse error verir
            fprintf(stderr, "Error: Program exceeds maximum line limit of %d.\n", MAX_PROGRAM_LINES);
            fclose(file);
            return 1;
        }
    }
    total_program_lines = current_line_in_file; //toplam satır sayısı atanması
    fclose(file); //dosya kapatılır

    //tüm satırlar belleğe yüklenince çağırılır
    execute_program_block(0, total_program_lines, 1);

    return 0;
}
```

The main function is the starting point of the interpreter. It handles command-line input or user prompt, opens the source file, reads it line by line, and starts execution.

- **Getting the File Name:** If the program is run with one argument (e.g., ./ppp myscript), it uses `snprintf` to safely append the .ppp extension to the input. If no argument is given, it prompts the user to type the filename and adds .ppp using `strcat`.
- **Opening the File:** The interpreter tries to open the file in read mode. If the file doesn't exist or can't be opened, it prints an error using `perror` and exits the program.
- **Reading the File into Memory:** Each line is read using `fgets`, which automatically null-terminates it. Then the line is trimmed to remove spaces and stored in the `all_program_lines` array. If the total line count exceeds `MAX_PROGRAM_LINES`, an error is shown and execution stops.
- **Tracking Total Lines:** After all lines are read, the total is stored in `total_program_lines`, and the file is closed with `fclose`.
- **Executing the Full Program:** Finally, the interpreter starts execution by calling `execute_program_block(0, total_program_lines, 1)`, which processes the entire program from the beginning, with accurate line tracking.

3) ERROR HANDLING

The interpreter includes built-in checks to detect and report common lexical and runtime errors defined by the Plus++ language rules. Each error message includes the specific line number for easy debugging.

- **Undeclared Variables:** Any attempt to assign or read a variable that was not declared using number results in an immediate error.
- **Invalid Identifiers:** Variable names must start with a letter and contain only letters, digits, or underscores. Invalid names are rejected.
- **Invalid Number Formats:** Inputs like --3, 12abc, or just - are flagged using `is_valid_number()`.
- **Syntax Errors:** The interpreter checks for missing semicolons, malformed assignment syntax, and improper use of `write`, `+=`, and `-=` operations.
- **Repeat Block Structure Errors:** Missing opening `{` or unmatched `}` in repeat loops are detected using brace-level tracking. Nested blocks are also validated.

All errors are printed to the console and stop program execution immediately using `exit(1)`, ensuring the user receives immediate feedback.

3.a) Undeclared Variable

Input:

```
1  
2  number sum;  
3  size:= 5;  
4  repeat size times  
5  {  
6    write size and newline;  
7    sum+=size;  
8  }  
9  write newline and "Sum:" and sum;
```

Output:

```
● ceylintor@192 pl_project2 2 % gcc -o ppp myscript.c  
⊗ ceylintor@192 pl_project2 2 % ./ppp myscript  
Error on line 3: Undeclared variable 'size'.
```

3.b) Invalid Variable Name

Input:

```
1  number 5size;
2  number sum;
3  size:= 5;
4  repeat size times
5  {
6      write size and newline;
7      sum+=size;
8  }
9  write newline and "Sum:" and sum;
```

Output:

```
● ceylintor@192 pl_project2 2 % gcc -o ppp myscript.c
⊗ ceylintor@192 pl_project2 2 % ./ppp myscript
Syntax error on line 1: Invalid 'number' declaration.
```

3.c) Missing Semicolon

Input:

```
1  number size
2  number sum;
3  size:= 5;
4  repeat size times
5  {
6      write size and newline;
7      sum+=size;
8  }
9  write newline and "Sum:" and sum;
```

Output:

```
● ceylintor@192 pl_project2 2 % gcc -o ppp myscript.c
⊗ ceylintor@192 pl_project2 2 % ./ppp myscript
Syntax error on line 1: Missing ';' at end of statement.
```

3.d) Invalid Assignment Format

Input:

```
1  number size;
2  number sum;
3  size:= ;
4  repeat size times
5  {
6  |  write size and newline;
7  |  sum+=size;
8  |
9  |  write newline and "Sum:" and sum;
```

Output:

```
● ceylintor@192 pl_project2 2 % gcc -o ppp myscript.c
⊗ ceylintor@192 pl_project2 2 % ./ppp myscript
Error on line 3: Invalid value ''. Must be a number or declared variable.
```

3.e) Missing { in Repeat Block

Input:

```
1  number size;
2  number sum;
3  size:= 5;
4  repeat size times
5  |
6  |  write size and newline;
7  |  sum+=size;
8  |
9  |  write newline and "Sum:" and sum;
```

Output:

```
⊗ ceylintor@192 pl_project2 2 % ./ppp myscript
Syntax error on line 4: Missing opening '{' for repeat block.
```


3.f) Missing } in Repeat Block

Input:

```
1  number size;
2  number sum;
3  size:= 5;
4  repeat size times
5  {
6  |  write size and newline;
7  |  sum+=size;
8
9  write newline and "Sum:" and sum;|
10
```

Output:

```
⊗ ceylinton@192 pl_project2 2 % ./ppp myscript
Syntax error on line 4: Missing closing '}' for repeat block.
```

3.g) Invalid Number Format

Input:

```
1  number size;
2  number sum;
3  size:= --5;
4  repeat size times
5  {
6  |  write size and newline;
7  |  sum+=size;
8  }
9  write newline and "Sum:" and sum;
10
```

Output:

```
⊗ ceylinton@192 pl_project2 2 % ./ppp myscript
Error on line 3: Invalid value '--5'. Must be a number or declared variable.
```

Input:

```
1  number size;
2  number sum;
3  size:= - 5;
4  repeat size times
5  {
6    write size and newline;
7    sum+=size;
8  }
9  write newline and "Sum:" and sum;
10
```

Output:

```
⊗ ceylintor@192 pl_project2 2 % ./ppp myscript
Error on line 3: Invalid value '- 5'. Must be a number or declared variable.
```

4) NO ERROR EXAMPLES

4.a) myscript.ppp

Input:

```
1  number ;
2  number sum;
3  size:= 5;
4  repeat size times
5  {
6    write size and newline;
7    sum+=size;
8  }
9  write newline and "Sum:" and sum;
10
```

Output:

```
● ceylintor@192 pl_project2 2 % gcc -o ppp myscript.c
● ceylintor@192 pl_project2 2 % ./ppp myscript
5
4
3
2
1

Sum:15
```

4.b) test_assign.ppp

Input:

```
number x;
number y;
x := 12345678;
y := x;
write "x:" and x and newline and "y:" and y;
```

Output:

```
● ceylintor@192 pl_project2 2 % ./ppp test_assign
x:12345678
y:12345678
```

4.c) test_math.ppp

Input:

```
number total;
total := 9999999;
total += 1;
total -= 9999998;
write "Total is:" and total and newline;
```

Output:

```
● ceylintor@192 pl_project2 2 % ./ppp test_math
Total is:2
```

4.d) test_nested.ppp

Input:

```
number i;
number j;
i := 2;
repeat i times
{
  j := 2;
  repeat j times
  {
    write "i:" and i and " j:" and j and newline;
    j -= 1;
  }
  i -= 1;
}
```

Output:

```
● ceylintor@192 pl_project2 2 % ./ppp test_nested
i:2 j:2

i:2 j:1

i:1 j:2

i:1 j:1
```

4.e) test_repeat.ppp

Input:

```
number counter;
number result;
number step;

counter := 3;
result := 10;
step := 2;

repeat counter times
{
  write "Current result is:" and result and newline;
  result += step;
  step += 1;
}

write newline and "Final result is:" and result and newline and "Final step is:" and step;
```

Output:

```
● ceylinton@192 pl_project2 2 % ./ppp test_repeat
Current result is:10

Current result is:12

Current result is:15

Final result is:19
Final step is:5
```

4.f) test_write.ppp

Input:

```
number count;
count := 3;
write "The count is:" and count and newline;
```

Output:

```
● ceylinton@192 pl_project2 2 % ./ppp test_write
The count is:3
```